CODE XPERT

ARM Cortex-M4 디바이스 프로그래밍

16과. DMA 제어

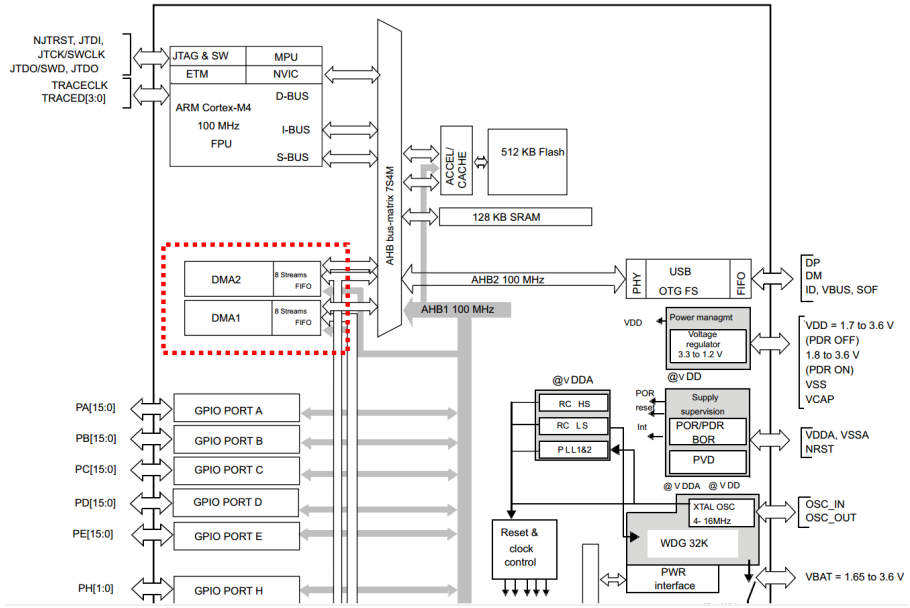
copyright© www.codexpert.co.kr

㈜윌텍
㈜행복한기술
㈜기쁜기술
㈜임베딕스
㈜에듀닉스

STM32F411 MCU의 DMA

CODE XPERT
임베디드 시스템 전문가그룹

❖ 2개의 DMA(Direct Memory Access)가 내부에 구성되어 있음



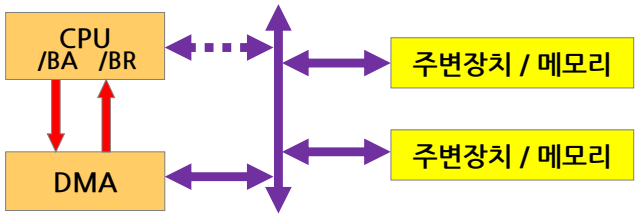
DMA의 동작

CODEXPERT 임베디드 시스템 전문가그룹

❖ CPU 관점에서 DMA도 주변장치 ➔ 즉, DMA도 CPU의 제어로 동작한다

- CPU는 외부메모리 액세스, 내부 연산 동작 등을 수행
- 내부 동작에서 외부 메모리 버스는 사용하지 않는다 ➔ 이 timing을 DMA가 사용

▪ [1] BR(Bus Request): DMA가 CPU에게 메모리 버스 제어권을 요구함
▪ [2] BA(Bus Ack): CPU가 내부 동작 timing에서 버스 제어권을 DMA에 양보
▪ [3] Transfer: DMA가 버스를 이용 메모리간 데이터 이동 (1 UNIT만 이동)
▪ [4] 제어권 반납: 1 UNIT만 이동한 후 다시 CPU에게 제어권을 반납
▪ [5] 정해진 크기의 메모리 이동을 마칠 때까지 [1] ~ [4] 과정을 계속 반복 수행
▪ DMA가 버스 사용(Cycle Steal)시 CPU는 버스 사용 불가 ➔ 최소 이동만 수행, 제어권 반납
▪ 즉, DMA는 최소한의 크기(1 UNIT)만 이동하고 다시 버스 제어권을 요청한다



 무단 복제, 전재, 발췌, 디지털화 금지

DMA 설정

CODEXPERT 임베디드 시스템 전문가그룹

❖ DMA로 메모리간 데이터 이동을 하려면 다음의 기본 정보 설정 필요

- 최소한의 동작을 위한 설정 정보는 다음과 같다

SOURCE 장치	→ DMA →	DESTINATION 장치
Source 시작 주소	이동방식(M2M,M2P,P2M)	Destination 시작 주소
한번 전송할 크기(UNIT)	전송할 횟수	한번 전송할 크기(UNIT)

- 다양한(고급) DMA 활용을 위하여 다음 설정이 추가적으로 필요 ➔ 매우 중요!
  - 단, 제조사 제품별로 DMA의 고급 설정은 다를 수 있다

SOURCE 장치	→ DMA →	DESTINATION 장치
주소 모드(증가/고정)	Trigger Source	주소 모드(증가/고정)
	Burst Size	
	Circular Mode	
	채널 우선순위 (4레벨)	

 무단 복제, 전재, 발췌, 디지털화 금지

주소 증가 모드란?

CODE XPERT 임베디드 시스템 전문가그룹

❖ [1] 증가 모드(INC): SRC, DEST가 동일한 크기로 데이터 이동

- 메모리 ➔ 메모리 등 일반적인 데이터 이동에 사용되는 모드
- 0x10번지(SRC) ➔ 0x1000번지(DEST)로 4B(UNIT)씩 50번(COUNT) 복사

SOURCE 메모리 (0x10 번지, 200B) → DMA → Destination 메모리 (0x1000 번지, 200B)

❖ [2] 고정 모드(FIX): SRC나 DEST가 데이터 이동시 주소를 고정시켜서 사용

- UART로 들어오는 수신 데이터 100B를 메모리에 복사한다면?
- UART(FIX) ➔ 메모리(INC), 1B 단위(UNIT)로 100번(COUNT)의 Transfer

- 메모리(버퍼)에 있는 100B의 데이터를 UART로 전송한다면?
- 메모리(INC) ➔ UART(FIX), 1B 단위(UNIT)로 100번(COUNT)의 Transfer

- 이와 같이 DMA가 동일한 주소에서 데이터를 읽거나 쓰는 경우 증가 모드 설정 필요
- SRC, DEST이 주소 모드에 따라 FIXed 혹은 INCrement로 설정되어야 한다

 무단 복제, 전재, 발췌, 디지털화 금지

Trigger Source

CODE XPERT 임베디드 시스템 전문가그룹

❖ DMA에 전송 정보를 모두 설정한 후 DMA 동작을 Start 시작시키는 동작 필요

- P2M, M2P의 경우 Peripheral의 동작이 준비되어야 DMA가 1 unit transfer가 가능하다
  - 이를 위하여 DMA에게 unit transfer trigger를 주는 장치(trigger source)의 지정을 해야 한다

- [설정 예] SRC 주소: 0x10, DEST 주소: 0x1000, UNIT: 4B, COUNT: 100
- 언제 DMA가 SRC의 내용을 DEST로 복사하게 되는가?
- DMA는 CPU에게 버스 요구, 제어권 인수를 100번(한번에 4B 복사) 반복해서 복사
- DMA가 이러한 동작을 Start 하도록 하는 행위를 Triggering이라고 한다

- 메모리-메모리 이동은 대부분 프로그램에서 DMA를 Start를 시킨다
- 이렇게 소프트웨어를 트리거 Source로 이용하는 경우를 SW 트리거 방식이라 한다

SOURCE 메모리 (0x10 번지, 400B) → DMA → Destination 메모리 (0x1000 번지, 400B)

Trigger Source의 Triggering

- Source가 UART 수신일 경우? UART에 데이터가 들어와야 DMA가 동작 가능
- 이런 경우 H/W(UART 수신 장치)가 Trigger Source가 된다 ➔ H/W 트리거

 무단 복제, 전재, 발췌, 디지털화 금지

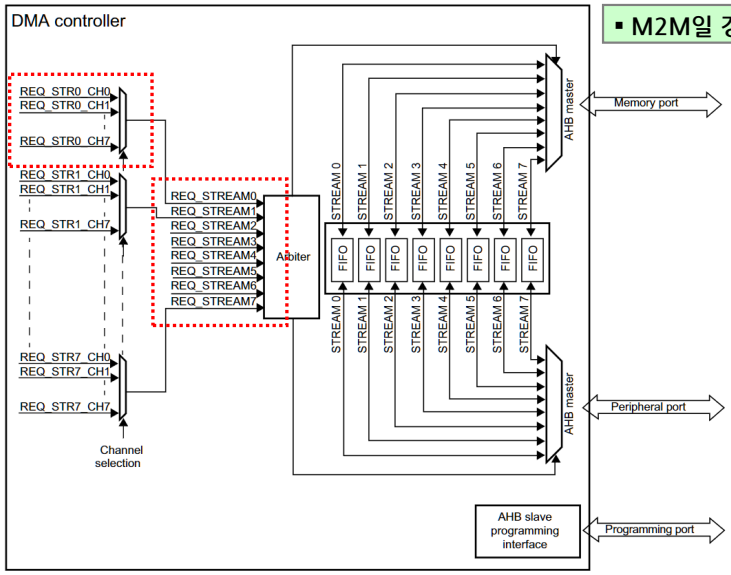
STM32F1xx의 DMA 구조

CODEXPERT 임베디드 시스템 전문가그룹

❖ DMA별로 8개 Stream, Stream별로 8개 Channel로 구성

- 하나의 DMA가 8개의 stream을 처리하여 8개의 DMA가 있는 것처럼 동작할 수 있다

▪ M2M일 경우 Channel은 의미 없음



DMA 동작 설정 : DMA stream x configuration register

CODEXPERT 임베디드 시스템 전문가그룹

❖ DMAx_Streamn->CR: 각 Stream에 대하여 DMA 동작을 위한 기본 설정

- CHSEL: 채널 선택(000 ~ 111) ➔ M2P, P2M에서 의미 있음
- MBURST / PBUSRT: Burst Unit 횟수(00:1 unit, 01:4, 10:8, 11:16)
- PL: 우선순위 (00: LOW, 01: Medium, 10: High, 11: Very High)
- MSIZE / PSIZE: mem측, peri측의 1 unit의 size (00:8b, 01:16b, 10:32b)
- MINC / PINC: mem측, peri의 주소 증가, 고정 모드 (0: 고정, 1: 증가)
- CIRC: Circular Mode enable(1), disable(0) ➔ Auto Reload Mode 단, M2M은 미지원
- DIR: Peri ➔ Mem(00), Mem ➔ Peri(01), Mem ➔ Mem(10)
- TCIE: Transfer Complete Interrupt enable(1), disable(0)
- HTIE: Half Transfer Interrupt enable(1), disable(0)
- EN: Enable Channel(1), disable(0)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				CHSEL[2:0]			MBURST [1:0]		PBURST[1:0]		Reser-ved	CT	DBM	PL[1:0]	
				rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PINCOS	MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]		PFCTRL	TCIE	HTIE	TEIE	DMEIE	EN
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

DMAx_Streamn→CR 설정

CODEXPERT 임베디드 시스템 전문가그룹

❖ DMA의 DIR(direction) 지정에 따른 DMA의 Source측, Peripheral측의 구분

- M2M일 경우 Source측은 Peripheral, Destination측은 Memory로 지정되어야 한다
  - 단, M2M(Memory-to-memory)는 DMA2에서만 지원됨

Bits DIR[1:0] of the DMA_SxCR register	Direction	Source address	Destination address
00	Peripheral-to-memory	DMA_SxPAR	DMA_SxM0AR
01	Memory-to-peripheral	DMA_SxM0AR	DMA_SxPAR
10	Memory-to-memory	DMA_SxPAR	DMA_SxM0AR

- 본 과정에서는 F411의 특수 기능보다는 보편적인 DMA 기능을 학습한다
  - CT, DBM: 하드웨어적으로 지원하는 double buffer 모드는 사용하지 않음
  - PBURST, MBURST: Burst 모드는 1 unit을 사용함
  - MSIZE, PSIZE : Source와 destination의 unit size는 서로 같은 값을 사용함
  - PINCOS: Peripheral측이 주소 증가 모드일 경우 증가하는 offset은 unit size만큼 씩 증가시킴

 무단 복제, 전재, 발췌, 디지털화 금지

주소와 횟수의 설정

CODEXPERT 임베디드 시스템 전문가그룹

❖ DMAx_Streamn->NDTR : 각 스트림의 Transfer Count를 설정(최대 65,535)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NDT															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

❖ DMAx_Streamn->M0AR: 각 스트림의 MEM 측 0번 주소를 저장

- Double buffer를 위하여 M0AR, M1AR 2개가 존재 ➔ 실습은 double buffer 사용 안 함

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MA																															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

❖ DMAx_Streamn->PAR : 각 스트림의 PERI 측 주소를 저장

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PA																															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

 무단 복제, 전재, 발췌, 디지털화 금지

Stream별 인터럽트 발생 상태 확인

CODEXPERT 임베디드 시스템 전문가그룹

❖ DMAx->LISR, DMAx->HISR: 8개 Stream에 대한 상태 Flag을 확인

- DMAx->LISR: Stream 0 ~ 3에 대한 인터럽트 flag

▪ TEIF(Transfer Error), DMEIF(Direct Mode Error), FEIF(FIFO Error) 발생(1)
▪ HTIF: Half Transfer 발생 (1), TCIF: Transfer Complete 발생 (1)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				TCIF3	HTIF3	TEIF3	DMEIF3	Reserved	FEIF3	TCIF2	HTIF2	TEIF2	DMEIF2	Reserved	FEIF2
r	r	r	r	r	r	r	r		r	r	r	r	r		r

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				TCIF1	HTIF1	TEIF1	DMEIF1	Reserved	FEIF1	TCIF0	HTIF0	TEIF0	DMEIF0	Reserved	FEIF0
r	r	r	r	r	r	r	r		r	r	r	r	r		r

- DMAx->HISR: Stream 4 ~ 7에 대한 인터럽트 flag

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				TCIF7	HTIF7	TEIF7	DMEIF7	Reserved	FEIF7	TCIF6	HTIF6	TEIF6	DMEIF6	Reserved	FEIF6
				r	r	r	r		r	r	r	r	r		r

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				TCIF5	HTIF5	TEIF5	DMEIF5	Reserved	FEIF5	TCIF4	HTIF4	TEIF4	DMEIF4	Reserved	FEIF4
				r	r	r	r		r	r	r	r	r		r

무단 복제, 전재, 발췌, 디지털화 금지

Stream별 인터럽트 발생 Flag Clear

CODEXPERT 임베디드 시스템 전문가그룹

❖ DMAx->LIFCR, DMAx->HIFCR : 8개 Stream에 대한 상태 Flag Clear

- DMAx->LIFCR: Stream 0 ~ 3에 대한 인터럽트 flag clear(1 쓰면 clear ➔ 대입연산)

▪ CTEIF(Transfer Error), CDMEIF(Direct Mode Error), CFEIF(FIFO Error) 발생 clear(1)
▪ CHTIF: Half Transfer 발생 (1), CTCIF: Transfer Complete 발생 clear(1)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				CTCIF3	CHTIF3	CTEIF3	CDMEIF3	Reserved	CFEIF3	CTCIF2	CHTIF2	CTEIF2	CDMEIF2	Reserved	CFEIF2
				w	w	w	w		w	w	w	w	w		w

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				CTCIF1	CHTIF1	CTEIF1	CDMEIF1	Reserved	CFEIF1	CTCIF0	CHTIF0	CTEIF0	CDMEIF0	Reserved	CFEIF0
				w	w	w	w		w	w	w	w	w		w

- DMAx->HLIFCR: Stream 4 ~ 7에 대한 인터럽트 flag clear(1 쓰면 clear ➔ 대입연산)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				CTCIF7	CHTIF7	CTEIF7	CDMEIF7	Reserved	CFEIF7	CTCIF6	CHTIF6	CTEIF6	CDMEIF6	Reserved	CFEIF6
				w	w	w	w		w	w	w	w	w		w

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				CTCIF5	CHTIF5	CTEIF5	CDMEIF5	Reserved	CFEIF5	CTCIF4	CHTIF4	CTEIF4	CDMEIF4	Reserved	CFEIF4
				w	w	w	w		w	w	w	w	w		w

무단 복제, 전재, 발췌, 디지털화 금지

Stream별 FIFO 제어 레지스터, DMA Clock ON 레지스터

CODEXPERT 임베디드 시스템 전문가그룹

❖ DMAx_Streamn->FCR: F411은 M2M 모드이면 반드시 FIFO를 사용해야 한다

- FEIE: FIFO Error Interrupt Enable (1)
- FS: FIFO Status (000: 1/4 이하, 001: 2/4 이하, 010: 3/4 이하, 011: 3/4 이상, 100: Empty, 101: Full)
- DMDIS: Direct Mode Disable (0: Enable, 1:Disable) ➔ M2M 모드로 설정하면 무조건 disable로 설정한다
- FTH: FIFO Threshold Selection (00: 1/4, 01: 2/4, 10: ¾, 11: 4/4)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								FEIE	Reserved	FS[2:0]			DMDIS	FTH[1:0]	
								rw		r	r	r	rw	rw	rw

❖ RCC->AHB4ENR 레지스터 : 사용하는 DMA의 Clock을 ON 시킨다

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									DMA2EN	DMA1EN	Reserved				
									rw	rw					

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved			CRCEN	Reserved				GPIOHEN	Reserved		GPIOEEN	GPIODEN	GPIOCEN	GPIOBEN	GPIOAEN
			rw					rw			rw	rw	rw	rw	rw

DMA 각 Stream별 인터럽트 번호

CODEXPERT 임베디드 시스템 전문가그룹

❖ DMA별로 다음과 같이 인터럽트 번호화 ISR 함수명이 배정되어 있음

- DMA2의 경우 Stream0 ~ Stream7의 인터럽트 번호가 연속하지 않으므로 주의해야 한다

IRQ 번호	종류	ISR 함수명
11	DMA1_Stream0	DMA1_Stream0_IRQHandler
12	DMA1_Stream1	DMA1_Stream1_IRQHandler
13	DMA1_Stream2	DMA1_Stream2_IRQHandler
14	DMA1_Stream3	DMA1_Stream3_IRQHandler
15	DMA1_Stream4	DMA1_Stream4_IRQHandler
16	DMA1_Stream5	DMA1_Stream5_IRQHandler
17	DMA1_Stream6	DMA1_Stream6_IRQHandler
47	DMA1_Stream7	DMA1_Stream7_IRQHandler
56	DMA2_Stream0	DMA2_Stream0_IRQHandler
57	DMA2_Stream1	DMA2_Stream1_IRQHandler
58	DMA2_Stream2	DMA2_Stream2_IRQHandler
59	DMA2_Stream3	DMA2_Stream3_IRQHandler
60	DMA2_Stream4	DMA2_Stream4_IRQHandler
68	DMA2_Stream5	DMA2_Stream5_IRQHandler
69	DMA2_Stream6	DMA2_Stream6_IRQHandler
70	DMA2_Stream7	DMA2_Stream7_IRQHandler

실습. SW에 의한 메모리 복사

CODE XPERT 임베디드 시스템 전문가그룹

❖ 메모리를 메모리로 복사하는 다음 프로그램의 동작을 분석하라

main.c

```
#define NUM_OF_DATA 40

unsigned int src1_dat[NUM_OF_DATA];
unsigned int dst1_dat[NUM_OF_DATA];

void Main(void)
{
   for(i=0;i<NUM_OF_DATA;i++)
   {
      src1_dat[i] = i + 8;
      dst1_dat[i] = 0;
   }

   Uart1_Printf("Memory Copy Test₩n₩n");
   for(i=0;i<NUM_OF_DATA;i++)
   {
      dst1_dat[i] = src1_dat[i];
   }
   for(i=0;i<NUM_OF_DATA;i++)
   {
      Uart1_Printf("dst1_dat[%d] = %d₩n",i,dst1_dat[i]);
      dst1_dat[i] = 0;
   }

   for(;;)
   {
      ...
   }
}
```

- SW적으로 메모리를 복사 수행
- 메모리 복사하는 동안 다른 일을 하지 못함

 무단 복제, 전재, 발췌, 디지털화 금지

DMA로 메모리 복사를 위한 DMA2 설정 함수 분석

CODE XPERT 임베디드 시스템 전문가그룹

❖ SW Trigger 동작을 위하여 DMA2 Stream0를 사용하도록 하는 함수 분석

- M2M은 DMA2만 지원하며 Stream은 0~7 임의로 선택할 수 있다

```
void DMA2_Stream0_M2M_Start(void * src_addr, void * dst_addr, int num)
{
  Macro_Set_Bit(RCC->AHB1ENR, 22);

  DMA2_Stream0->CR = (0x0<<25)|(0x2<<13)|(0x2<<11)|(0x1<<10)|(0x1<<9)|(0x2<<6)|(0x0<<0);
  DMA2_Stream0->FCR = (0x1<<2)|(0x3<<0);
  DMA2_Stream0->PAR  = (unsigned int)src_addr;
  DMA2_Stream0->M0AR = (unsigned int)dst_addr;
  DMA2_Stream0->NDTR = num;

  DMA2->LIFCR = 0x3F << 0;
  Macro_Set_Bit(DMA2_Stream0->CR, 4);
  NVIC_ClearPendingIRQ(56);
  NVIC_EnableIRQ(56);

  Macro_Set_Bit(DMA2_Stream0->CR, 0);
}
```

dma.c

 무단 복제, 전재, 발췌, 디지털화 금지

실습. DMA에 의한 메모리 복사

CODE XPERT 임베디드 시스템 전문가그룹

❖ DMA로 전송하는 동안 실시간 프로그램 수행이 가능함

```c
#define NUM_OF_DATA     40
unsigned int src1_dat[NUM_OF_DATA];
unsigned int dst1_dat[NUM_OF_DATA];
extern volatile int DMA2_STREAM0_DONE;

void Main(void)
{
    for(i = 0; i < NUM_OF_DATA; i++) { ... }

    DMA2_Stream0_M2M_Start(src1_dat, dst1_dat, NUM_OF_DATA);

    for(;;)
    {
        if(DMA2_STREAM0_DONE)
        {
            printf("\nDMA2-Stream0 Complete!\n");
            DMA2_STREAM0_DONE = 0;

            for(i = 0; i < NUM_OF_DATA; i++) printf("@dst1_dat[%d] = %d\n", i, dst1_dat[i]);
        }

        if(TIM4_Expired) { ... }
    }
}
```

main.c

DMA 종료 인터럽트가 발생하면 복사된 내용 인쇄

```c
volatile int DMA2_STREAM0_DONE = 0;

void DMA2_Stream0_IRQHandler(void)
{
  DMA2->LIFCR = 0x3F << 0;
  NVIC_ClearPendingIRQ(56);
  DMA2_STREAM0_DONE = 1;
}
```

Exception.c

 무단 복제, 전재, 발췌, 디지털화 금지

실습. printf 함수를 이용한 UART2에 문자열 인쇄

CODE XPERT 임베디드 시스템 전문가그룹

❖ USART2로 문자열을 인쇄하는 다음 프로그램의 동작을 분석하라

main.c

```c
extern volatile int TIM4_Expired;

const char * str[] = {"문장1", "문장2", ... "문장9"};

void Main(void)
{
    unsigned int str_num = 0;

    for (;;)
    {
      if(str_num < (sizeof(str)/sizeof(str[0])))
      {
        printf("%s", str[str_num]);
        str_num++;
      }

      if(TIM4_Expired)
      {
        ...
      }
    }
}
```

인쇄하는 동안 실시간성 훼손

 무단 복제, 전재, 발췌, 디지털화 금지

DMA1 Stream별 채널 할당

CODEXPERT 임베디드 시스템 전문가그룹

❖ Stream별로 Trigger로 사용하는 주변장치는 채널별로 지정되어 있다

Peripheral requests	Stream 0	Stream 1	Stream 2	Stream 3	Stream 4	Stream 5	Stream 6	Stream 7
Channel 0	SPI3_RX	I2C1_TX	SPI3_RX	SPI2_RX	SPI2_TX	SPI3_TX	-	SPI3_TX
Channel 1	I2C1_RX	I2C3_RX	-	-	-	I2C1_RX	I2C1_TX	I2C1_TX
Channel 2	TIM4_CH1	-	I2S3_EXT_RX	TIM4_CH2	I2S2_EXT_TX	I2S3_EXT_TX	TIM4_UP	TIM4_CH3
Channel 3	I2S3_EXT_RX	TIM2_UP TIM2_CH3	I2C3_RX	I2S2_EXT_RX	I2C3_TX	TIM2_CH1	TIM2_CH2 TIM2_CH4	TIM2_UP TIM2_CH4
Channel 4	-	-	-	-	-	USART2_RX	USART2_TX	-
Channel 5	-	-	TIM3_CH4 TIM3_UP	-	TIM3_CH1 TIM3_TRIG	TIM3_CH2	-	TIM3_CH3
Channel 6	TIM5_CH3 TIM5_UP	TIM5_CH4 TIM5_TRIG	TIM5_CH1	TIM5_CH4 TIM5_TRIG	TIM5_CH2	I2C3_TX	TIM5_UP	USART2_RX
Channel 7	-	-	I2C2_RX	I2C2_RX	-	-	-	I2C2_TX

무단 복제, 전재, 발췌, 디지털화 금지

DMA2 Stream별 채널 할당

CODEXPERT 임베디드 시스템 전문가그룹

❖ Stream별로 Trigger로 사용하는 주변장치는 채널별로 지정되어 있다

- M2M은 DMA2만 가능하다(Stream은 0~7 모두 사용 가능)

Peripheral requests	Stream 0	Stream 1	Stream 2	Stream 3	Stream 4	Stream 5	Stream 6	Stream 7
Channel 0	ADC1	-	-	-	ADC1	-	TIM1_CH1 TIM1_CH2 TIM1_CH3	-
Channel 1	-	-	-	-	-	-	-	-
Channel 2	-	-	SPI1_TX	SPI5_RX	SPI5_TX	-	-	-
Channel 3	SPI1_RX	-	SPI1_RX	SPI1_TX	-	SPI1_TX	-	-
Channel 4	SPI4_RX	SPI4_TX	USART1_RX	SDIO	SPI4_RX	USART1_RX	SDIO	USART1_TX
Channel 5	-	USART6_RX	USART6_RX	SPI4_RX	SPI4_TX	SPI5_TX	USART6_TX	USART6_TX
Channel 6	TIM1_TRIG	TIM1_CH1	TIM1_CH2	TIM1_CH1	TIM1_CH4 TIM1_TRIG TIM1_COM	TIM1_UP	TIM1_CH3	-
Channel 7	-	-	-	-	-	SPI5_RX	SPI5_TX	-

무단 복제, 전재, 발췌, 디지털화 금지

DMA로 USART2 TX를 위한 DMA1 설정 함수 분석

CODE X PERT 임베디드 시스템 전문가그룹

❖ USART2 TX를 위한 DMA1, Stream 6번, Channel 4 설정 함수 분석

```
void DMA1_Stream6_USART2_TX_Satrt(void * src_addr, int num)
{
  Macro_Set_Bit(RCC->AHB1ENR, 21);

  DMA1_Stream6->CR = (0x4<<25)|(0x0<<13)|(0x0<<11)|(0x1<<10)|(0x0<<9)|(0x1<<6)|(0x0<<0);
  DMA1_Stream6->FCR = 0;
  DMA1_Stream6->PAR  = (unsigned int)&USART2->DR;
  DMA1_Stream6->M0AR = (unsigned int)src_addr;
  DMA1_Stream6->NDTR = num;

  DMA1->HIFCR = 0x3F << 0;
  Macro_Set_Bit(DMA1_Stream6->CR, 4);
  NVIC_ClearPendingIRQ(15);
  NVIC_EnableIRQ(15);

  Macro_Set_Bit(DMA1_Stream6->CR, 0);
}
```

dma.c

 무단 복제, 전재, 발췌, 디지털화 금지

실습. H/W 트리거 DMA 동작

CODE X PERT 임베디드 시스템 전문가그룹

❖ USART2 TX로 한 문장을 DMA로 전송하는 다음 코드의 동작을 분석하라

```
const char * str[] = {"문장1", "문장2", ... "문장9"};
extern volatile int DMA1_STREAM6_DONE;

void Main(void)
{
    unsigned int str_num = 0;
    Uart2_Wait_for_TX_Complete();

    for (;;)
    {
        if (DMA1_STREAM6_DONE || (str_num == 0))
        {
            if (str_num < (sizeof(str) / sizeof(str[0])))
            {
                Macro_Set_Bit(USART2->CR3, 7);
                DMA1_Stream6_USART2_TX_Satrt((void * )str[str_num], strlen(str[str_num]));
                str_num++;
                DMA1_STREAM6_DONE = 0;
            }

            else
            {
                Macro_Clear_Bit(DMA1_Stream6->CR, 0);
                Macro_Clear_Bit(USART2->CR3, 7);
            }
        }

        if(TIM4_Expired) { ... }
    }
}
```

main.c

DMA가 알아서 문자열 전송
문장을 전송하는 동안 다른 업무 가능

```
volatile int DMA1_STREAM6_DONE = 0;

void DMA1_Stream6_IRQHandler(void)
{
  DMA2->HIFCR = 0x3F << 0;
  NVIC_ClearPendingIRQ(15);
  DMA1_STREAM6_DONE = 1;
}
```

Exception.c

 무단 복제, 전재, 발췌, 디지털화 금지

TC와 TXE Flag의 차이

CODEXPERT 임베디드 시스템 전문가그룹

❖ UART는 대부분 TX Buffer Empty와 TX Complete Flag이 별도로 있다

- USART1→SR: TXE(TX Buffer Empty), TC(Transmit Complete)
  - Uart1_Printf 함수는 TXE가 발생하면 마지막 글자를 버퍼에 복사한 후 리턴 한다
  - 그러나 UART는 시리얼로 비트 단위 전송을 하므로 Uart1_Printf 종료후에도 전송을 한다
  - 따라서 UART 전송이 완전히 끝났는지를 판단하려면 TC Flag을 판단하여야 한다

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						CTS	LBD	TXE	TC	RXNE	IDLE	ORE	NE	FE	PE
						rc_w0	rc_w0	r	rc_w0	rc_w0	r	r	r	r	r

- TC Flag을 확인해야 하는 경우
  - 데이터 전송 완료 후 UART 전원이나 Clock을 차단하는 경우
  - Half Duplex 방식에서 송신 후 바로 수신모드로 전환하는 경우
  - DMA를 이용하여 UART를 사용하는 경우 ➔ 이전 인쇄를 완전 종료한 후 DMA 개시 필요

Main.c

```
void Uart1_Wait_for_TX_Complete(void)
{
  while(Macro_Check_Bit_Clear(USART1->SR, 6));
}
```


무단 복제, 전재, 발췌, 디지털화 금지

MEMO

CODEXPERT 임베디드 시스템 전문가그룹

무단 복제, 전재, 발췌, 디지털화 금지

CODEXPERT

ARM Cortex-M4 디바이스 프로그래밍

17과. CAN 통신

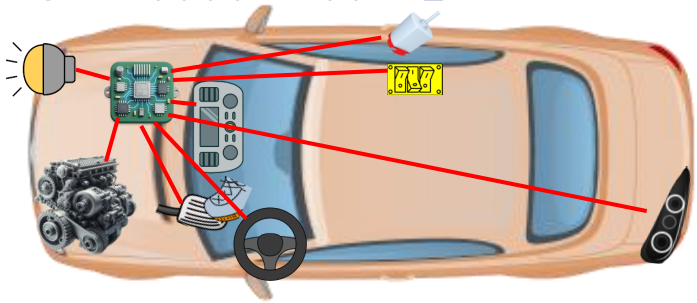
copyright© www.codexpert.co.kr

㈜웹텍
㈜행복한기술
㈜기쁜기술
㈜임베딕스
㈜에듀닉스

자동차의 전장 구성

CODEXPERT
임베디드 시스템 전문가그룹

❖ 고전적인 방식 : 주제어기에 모든 제어 요소를 전선으로 연결



- Harness Wire 예와 자동차의 Harness Wire로 가득 찬 차체



출처: https://blog.naver.com/datagram2501/223732009853



CAN의 등장

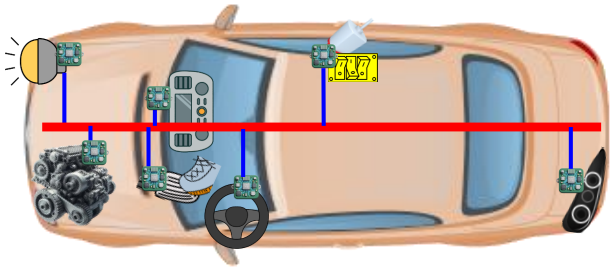
CODEXPERT 임베디드 시스템 전문가그룹

❖ Harness Wire를 Network로 묶어서 배선을 간략히 할 수는 없을까?

- Main Controller가 집중 제어 ➔ 통신에 연결된 각 노드들로 분산제어
  - 각 노드들이 MCU 보유(모듈화)한 ECU(Electronics Control Unit)가 된다
- 소프트웨어의 변경 없이도 새로운 기능의 ECU 추가, 삭제가 자유롭게
  - (예) 초기 모델은 일반 헤드라이트를 탑재 ➔ 업그레이드 모델은 적응형 헤드라이트 탑재
- 동작 중 어떤 ECU가 고장나는 경우 네트워크에서 배제 시키고 동작 가능하게

❖ Bosch에서 CAN Bus Communication 방식 발표

- 자동차는 수백개 이상의 ECU 모듈들로 구성됨



[차량의 주요 구성]
- Power Train: 엔진, 미션, 변속기 등
- 차체(샤시) : 도어, 시트, 라이트 등
- 인포테인먼트 : AVN, 멀티미디어 등

- 신뢰성
- 분산처리
- 편리한 기능 요구(ECU 증가) 만족
- 무게 감소
- 배선 비용 절감 및 원가 절감

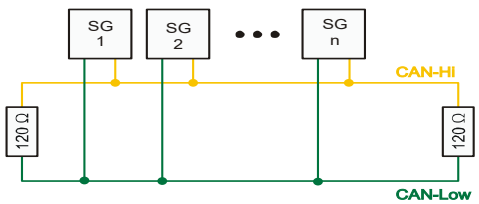
무단 복제, 전재, 발췌, 디지털화 금지

CAN의 간략한 역사와 특징

CODEXPERT 임베디드 시스템 전문가그룹

❖ CAN 특징 요약

- CSMA-CD/CR, 멀티마스터, 메시지 기반, Filtering
- CAN(Controller Area Network) 역사와 표준
  - 1980년대 Bosch사에서 CAN 프로토콜 발표
  - 1991년 CAN 2.0 발표(이 버전을 일반적으로 CAN이라 부름)
  - 2012년 CAN FD(Flexible Data Rate) 발표 ➔ 고속 데이터 대응
  - 2020년 CAN XL(eXtra Lond) 발표 ➔ 멀티미디어, 인터넷 데이터 대응
- 최대 1Mbps, 장거리(40kbps로 최대 1km), 비동기 직렬 통신, 차동 신호 방식
  - CAN은 자동차 뿐 아니라 다양한 산업 및 자동화 등에도 활용되고 있음
  - 연관 용어 : LIN, TTCAN, MOST, CANopen, CANoe 등



출처: https://en.wikipedia.org/wiki/CAN_bus#

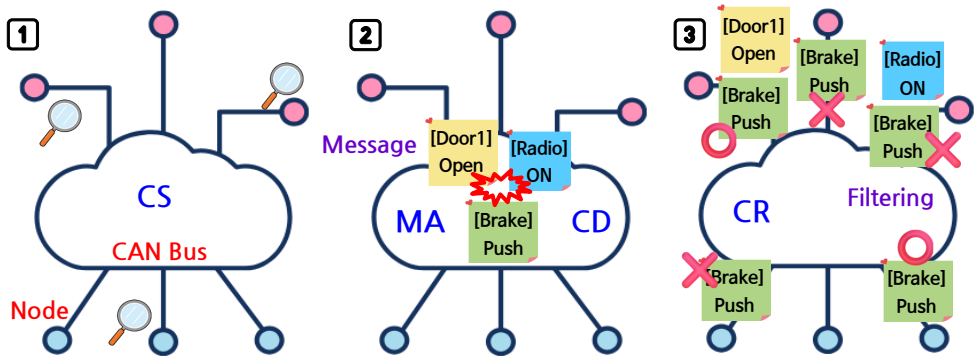
무단 복제, 전재, 발췌, 디지털화 금지

CAN의 핵심 동작 개념

CODE XPERT 임베디드 시스템 전문가그룹

❖ 멀티마스터, CSMA-CD/CR, 메시지 기반 Broadcasting & Filtering

- 멀티마스터 & CSMA: 마스터 없이 누구나 망이 비어 있으면 전송 가능
  - Carrier Sense(망 비어 있는지 감시) + Multiple Access(누구나 동시에 망 사용 가능)
  - Collision Detection(전송 중 충돌 확인) + Collision Resolution(충돌을 스스로 해결)
- 메시지 기반 Broadcasting: 수신인 지정 없이 메시지 전달 ➔ 모든 노드가 수신
  - Filtering: 원하지 않는 정보(발신자 확인)는 제거, 필요한 메시지만 받아서 처리

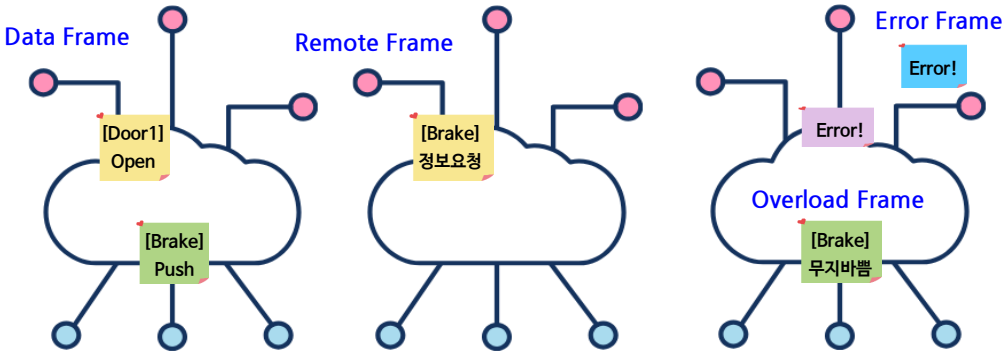


메시지 종류와 에러 처리

CODE XPERT 임베디드 시스템 전문가그룹

❖ CAN 통신 Frame

- 최대 메시지 개수 : Standard CAN(2.0A) : 211개, Extended CAN(2.0B) : 229개
  - 통신 frame에 ID 필드에 의하여 메시지 종류를 표시
  - 노드 수와 메시지 수는 무관 : 한 노드가 여러 종류 메시지를 발송할 수 있음
- 종류 : Data(전체 발송), Remote(메시지 요청), Overload, Error

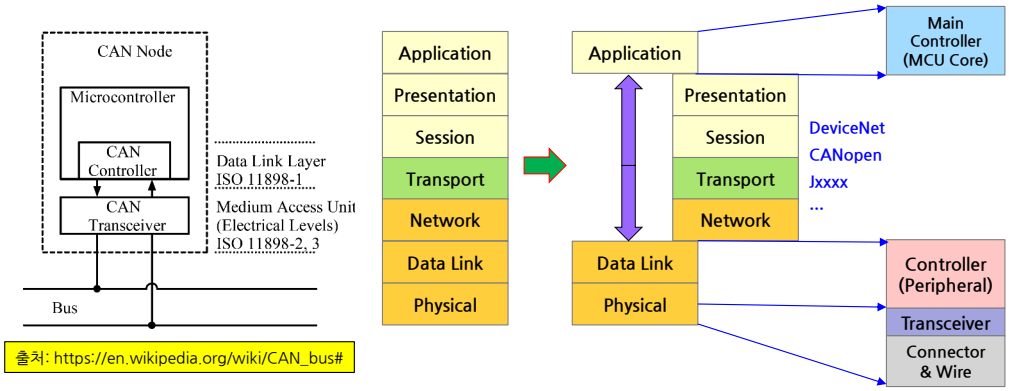


ISO-OSI 7L RM과 CAN Protocol

CODEXPERT 임베디드 시스템 전문가그룹

❖ CAN 2.0의 Protocol 표준

- ISO11898-1 ➔ Data Link Layer, ISO11898-2 & 3 ➔ Physical Layer
  - ISO11898-2(High Speed CAN) : 최대 1Mbps
  - ISO11898-3(Low Speed CAN) : 최대 125Kbps
- CAN을 이용하여 네트워크를 구현하는 많은 중간 계층 프로토콜 들이 존재한다



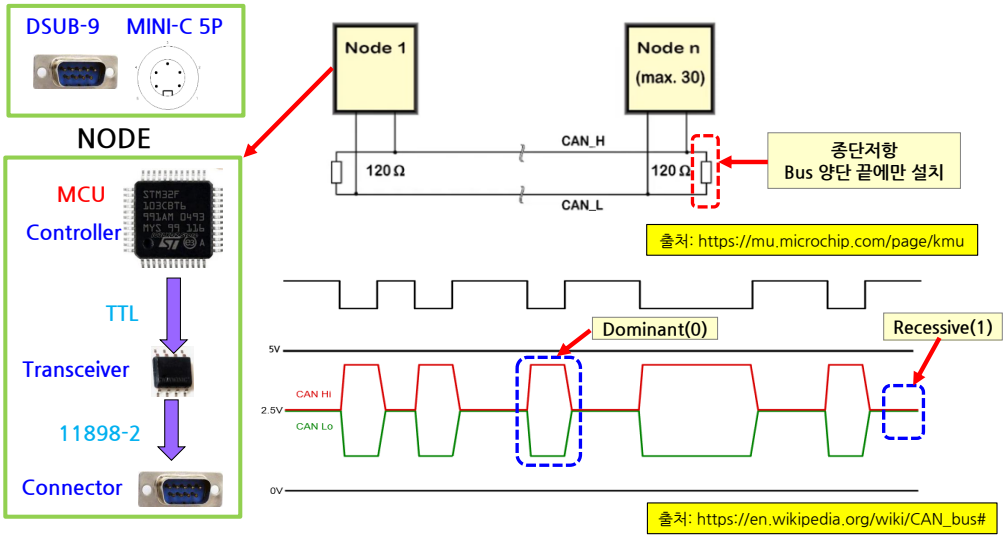
출처: https://en.wikipedia.org/wiki/CAN_bus#

ISO11898-2의 Modulation Signal

CODEXPERT 임베디드 시스템 전문가그룹

❖ ISO11898-2(High Speed CAN)의 네트워크 구성과 신호 규약

- 주로 Power Train 계통의 네트워크 구성에 사용된다



출처: https://mu.microchip.com/page/kmu

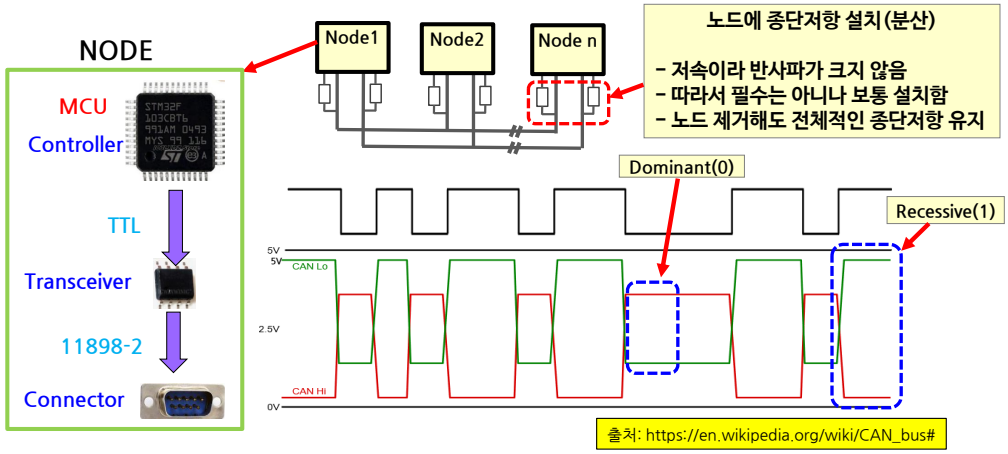
출처: https://en.wikipedia.org/wiki/CAN_bus#

ISO11898-3의 Modulation Signal

CODEXPERT 임베디드 시스템 전문가그룹

❖ ISO11898-3(Low Speed CAN)의 네트워크 구성과 신호 규약

- 주로 Power Train 이외의 샤시(Bosy) 계통의 네트워크 구성에 사용된다
  - Bus 뿐 아니라 star 등 다양한 topology로 구성이 가능하다
  - Fault Tolerant: Can-H나 Can-L가 끊어져도 선 하나로 Low, High 판별이 가능하다

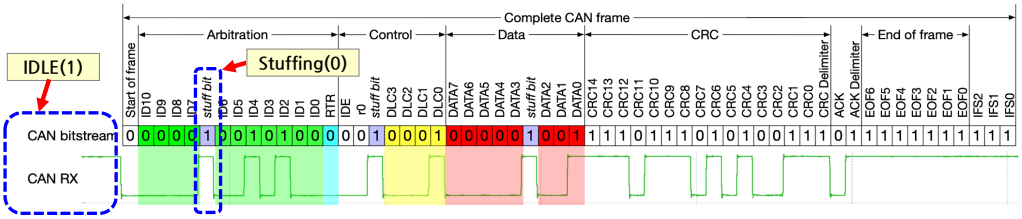


출처: https://en.wikipedia.org/wiki/CAN_bus#

Basic Data Frame(CAN 2.0A)

CODEXPERT 임베디드 시스템 전문가그룹

❖ Basic Data Frame(ID가 11비트(211개) 메시지)의 형식



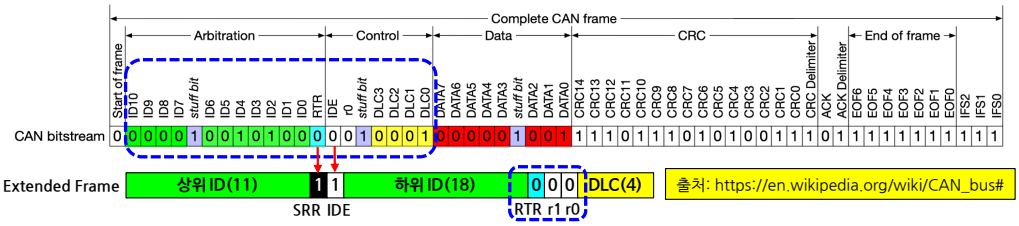
이름	길이(bits)	값	의미(Basic Data Frame)
Start-of-frame	1	0	
Identifier (green)	11	-	Message ID
Remote transmission request (RTR) (blue)	1	-	Data Frame(0), Remote Frame (1)
Identifier extension bit (IDE)	1	0	Base Frame (0), Extended Frame(1)
Reserved bit (r0)	1	0	
Data length code (DLC) (yellow)	4	-	Number of bytes of data (0 - 8)
Data field (red)	0-8 bytes	-	전송 데이터(DLC 지정 크기에 따름)
CRC	15	-	에러 발견 및 자가 복구용 CRC 정보
CRC delimiter	1	1	
ACK slot	1	-	수신한 노드들이 0 생성, 만약 1이면 Error
ACK delimiter	1	1	
End-of-frame (EOF)	7	1	Stuffing 없이 7비트 모두 1
Inter-frame spacing (IFS)	3	1	다음 Frame 생성 전 여유 기간

출처: https://en.wikipedia.org/wiki/CAN_bus#

Extended Data Frame(CAN 2.0B)

CODEXPERT 임베디드 시스템 전문가그룹

❖ Extended Data Frame(ID가 29비트(229개) 메시지)의 형식



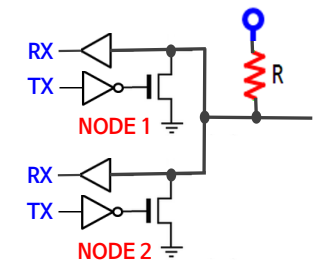
이름	길이(bits)	값	의미(Extended Data Frame)
Start-of-frame	1	0	
Identifier A (green)	11	-	Message ID의 상위 11비트
Substitute Remote Request (SRR) (black)	1	1	
Identifier extension bit (IDE)	1	1	Base Frame (0), Extended Frame(1)
Identifier B (green)	18	-	Message ID의 하위 18비트
Remote transmission request (RTR) (blue)	1	0	Data Frame(0), Remote Frame (1)
Reserved bits (r1, r0)	2	0	
Data length code (DLC) (yellow)	4		Number of bytes of data (0 - 8)
Data field (red)	0-8 bytes	-	전송 데이터(DLC 지정 크기에 따름)
CRC	15		에러 발견 및 자가 복구용 CRC 정보
CRC delimiter	1	1	
ACK slot	1	-	수신한 노드들이 0 생성, 만약 1이면 Error
ACK delimiter	1	1	
End-of-frame (EOF)	7	1	Stuffing 없이 7비트 모두 1
Inter-frame spacing (IFS)	3	1	다음 Frame 생성 전 여유 기간

CSMA-CD/CR와 Bus Arbitration

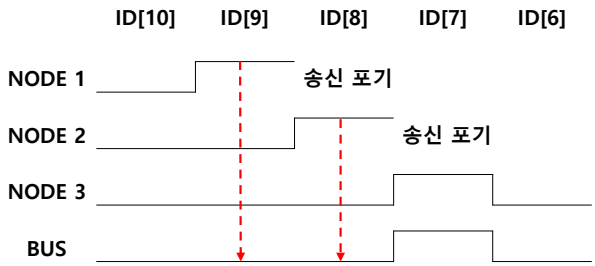
CODEXPERT 임베디드 시스템 전문가그룹

❖ BUS가 ILDE(1)이면 메시지 전송 ➔ 동시 전송 시 충돌 발생

- ID값이 작을수록 높은 우선순위 메시지로 취급된다 ➔ all '0'가 가장 높은 순위
  - CAN-L, CN-H 신호를 Open-Drain으로 구성하여 기본이 1이고 0을 쓰면 버스는 0이 된다
    - 따라서 1은 Recessive(열성)이라 하고 0은 Dominant(우성)이라고 한다
  - 모든 노드는 송신하는 데이터를 local loop back으로 수신하여 값이 일치하는지 비교한다
  - 즉, 1을 보냈는데 0이 읽혀지면 다른 우선 순위 높은 메시지와 충돌한 것으로 판단한다
    - 충돌을 감지한 노드는 즉시 수신 모드 노드로 전환되고 충돌전까지 보낸 정보를 수신 정보로 간주한다



CAN은 Open Drain으로 구성

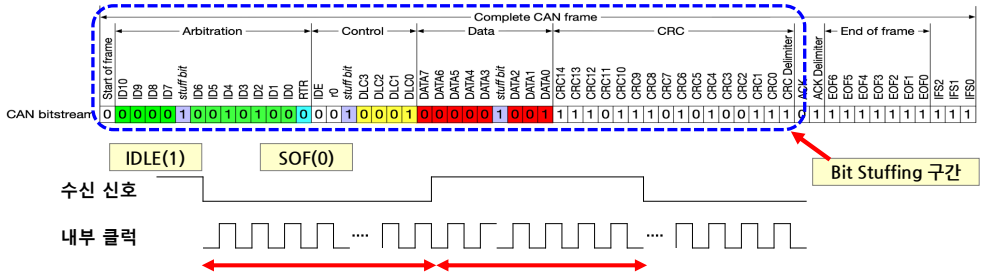


비동기 직렬 통신 : 동기화와 Bit Stuffing

CODEXPERT 임베디드 시스템 전문가그룹

❖ 비동기 통신 : 각 노드에서 데이터 추출을 위한 클럭 생성

- 각 노드들의 클럭 동기화 ➔ 수신 노드들이 송신 노드의 주파수에 맞춰야 한다
  - 통신 시작점 : Idle 상태에서 SOF가 발생하는 시점에 클럭 동기(synchronization) 수행
  - 0 ➔ 1, 1 ➔ 0 변화 시점 : 내부 클럭(고속) 미세 조절로 동기화 수행(resynchronization)



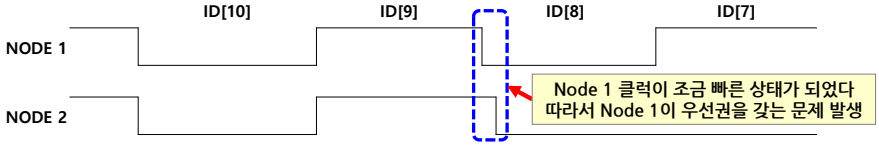
- Bit Stuffing에 의한 동기 유실 방지
  - 0 또는 1이 계속 송신되면? 수신 노드들의 동기화 불가 ➔ 데이터 수신 miss 발생 확률 up
  - 따라서 0이 연속 5개면 1, 1이 연속 5개면 0을 뒤에 삽입(stuffing) ➔ 수신 노드에서 제거
  - 단, stuffing은 SOF 비트부터 CRC 비트 구간에서만 추가된다

 무단 복제, 전재, 발췌, 디지털화 금지

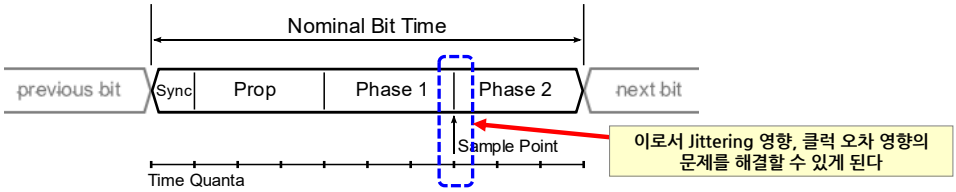
통신 속도 차이에 의한 우선순위 역전 문제

CODEXPERT 임베디드 시스템 전문가그룹

❖ 두 노드 송신 시 클럭이 차이가 있다면 어떤 문제가 있을 수 있을까?



- 이를 해결하기 위한 방법은? Edge 부분이 아닌 안정된 신호 부분에서 신호 감지
  - 약속한 bps를 고려하여 Sample Point에서 신호 판단
  - Next Bit(비트 값이 바뀌는 경우)에서 신호 반전이 되면 Time Quanta 개수 재조정(동기화)



출처: https://en.wikipedia.org/wiki/CAN_bus#

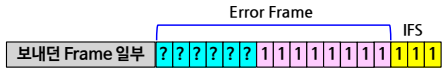
 무단 복제, 전재, 발췌, 디지털화 금지

통신 오류의 발견과 Error Frame

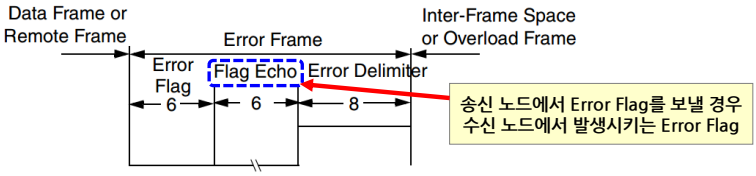
CODE XPERT 임베디드 시스템 전문가그룹

❖ 통신 오류가 발견되면 발견한 노드들이 그 즉시 Error Frame을 발생시킨다

- 6비트의 Error Flag와 8비트의 1 (Error Delimiter)로 구성된다
  - Active Error ➔ Error Flag = 6비트 0, Passive Error ➔ Error Flag = 6비트 1



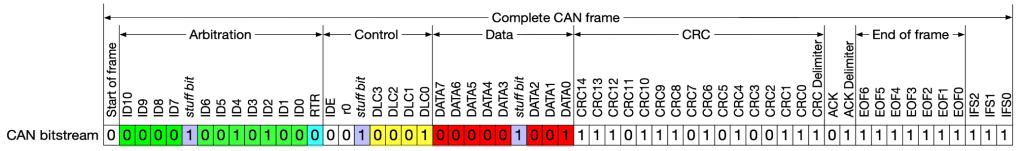
- 송신 노드가 오류를 발생하여 Error Frame을 보낸다면?
  - 수신 노드들은 연속한 6개의 0 또는 1을 수신하게 되어 Bit Stuffing 오류도 인식한다
    - 마지막 6번째 비트에서 Bit Stuffing 오류를 인지하므로 바로 이어서 Error Flag을 전송하게 된다
    - 이로 인하여 송신 노드가 발생시킨 Error Flame 뒤에 수신 노드들의 Error Flag가 추가되게 된다
    - 이렇게 Error Flag 뒤에 연속하여 발생되는 Error Flag를 Flag Echo라고 한다



통신 오류의 종류

CODE XPERT 임베디드 시스템 전문가그룹

❖ 발생하는 오류의 종류는 다음과 같다



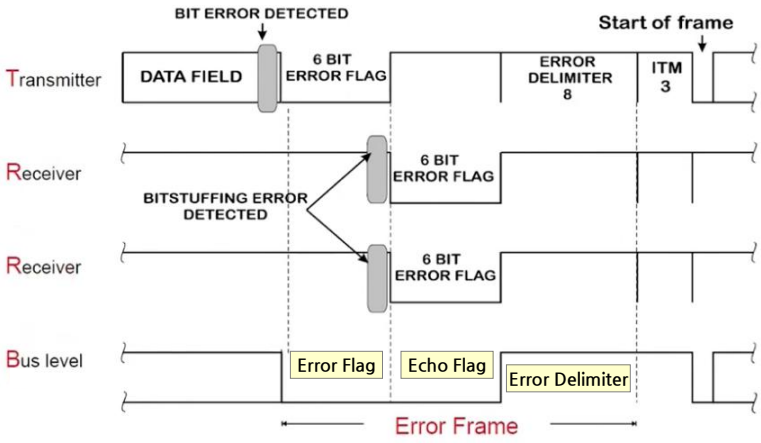
- Bit Error: 송신 노드가 자신이 보낸 값과 Loop-Back으로 받은 값이 다른 경우
  - 송신 노드에서 발견하게 되며 ACK와 Arbitration (ID 발송) 구간에서는 발생하지 않는다
- ACK Error: 송신 노드에서 ACK 구간에서 ACK가 감지되지 않을 경우
- Stuff Error: 1 또는 0이 연속 6개째 발생 ➔ SOF부터 CRC 범위에서만 유효함
  - 따라서 Frame 중간에 송신 노드가 발생시킨 Error Flag는 Stuffing Error로 간주되게 된다
- Form Error: 각 영역의 약속한 비트 형식이 맞지 않는 경우
  - EOF, IFS, ACK delimiter, CRC delimiter에 0이 발견될 경우
- CRC Error: 약속한 CRC 값이 틀릴 경우 (받은 CRC 값 ! = 직접 계산한 CRC 값)

Error Flag 발생 상황 검토 – 1

CODEXPERT 임베디드 시스템 전문가그룹

❖ 송신 노드 ➔ Bit Error 감지, 수신 노드 ➔ Stuffing Error 감지

- 이러한 경우 Error Flag가 반복(Echo Flag)되어 발생하게 된다



출처: https://mu.microchip.com/page/kmu

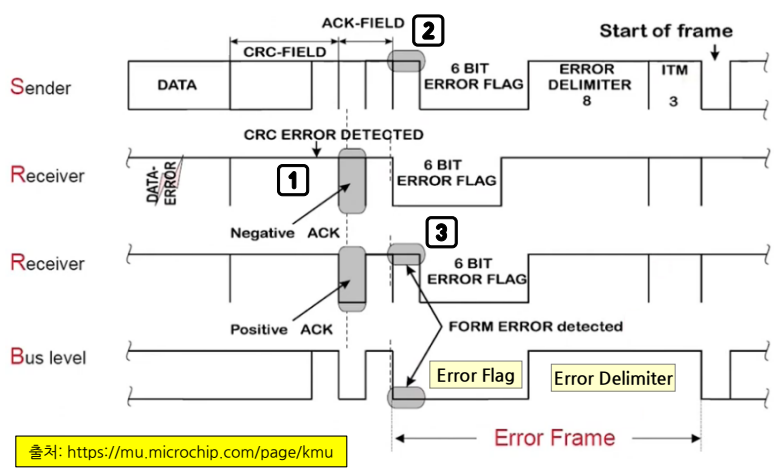
 무단 복제, 전재, 발췌, 디지털화 금지

Error Flag 발생 상황 검토 – 2

CODEXPERT 임베디드 시스템 전문가그룹

❖ 수신 노드 ➔ CRC Error 감지, 송신 노드 ➔ Form Error 감지

- 수신 노드들 중 하나의 노드가 CRC Error가 발생하였다면?
  - 다른 정상 수신 노드의 ACK로 정상으로 되어 문제가 발생하지 않는다
  - CRC 오류 감지 노드가 EOF 구간에 Error Flag 전송 ➔ 다른 노드들은 Form 에러로 인지



출처: https://mu.microchip.com/page/kmu

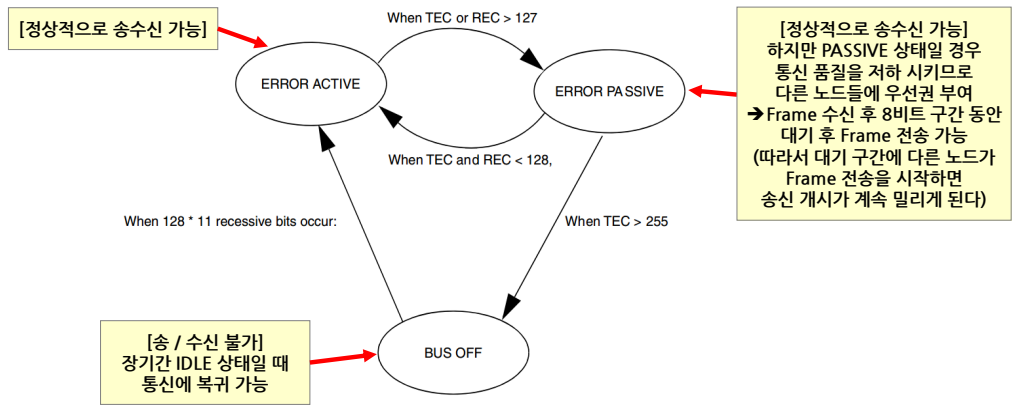
 무단 복제, 전재, 발췌, 디지털화 금지

Error State Diagram

CODEXPERT 임베디드 시스템 전문가그룹

❖ 노드들은 TEC(송신 오류 카운터), REC(수신 오류 카운터)를 갖는다

- 오류 발견시마다 카운트를 UP하고 정상 동작하면 카운트를 DOWN한다
  - 이 카운터의 값에 따라 다음과 같이 노드들의 상태가 천이 된다
    - Active Error: 스스로 복구할 수 있는 수준의 오류

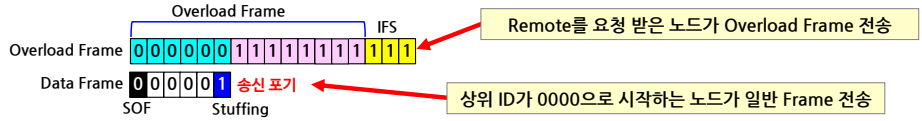


Overload Frame

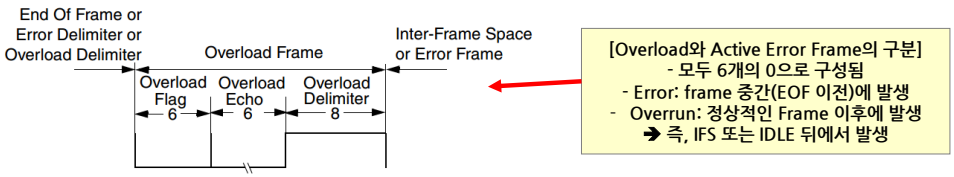
CODEXPERT 임베디드 시스템 전문가그룹

❖ Remote로 요청한 메시지를 발생시키는 노드가 바쁠 경우 보내는 Frame

- 이 메시지 Frame은 정상적으로 Remote Frame 수신된 후 최대한 빠르게 발송한다
  - Overload frame은 SOF와 ID없이 6비트의 0, 8비트의 1로 구성된다(bit stuffing 없음)
  - 만약 어떤 노드의 ID가 all '0'이어도 SOF와 + 상위 ID 4비트(0000)에 의해 stuffing 추가
  - 따라서 Overload Frame은 가장 높은 우선순위를 갖게 된다



- Overload를 보내면 수신 노드들은 Stuffing 에러로 판단하게 되어 Error Flag 발생
  - 따라서 Overload Flag를 보내면 Overload Echo가 추가되어 다음과 같이 된다



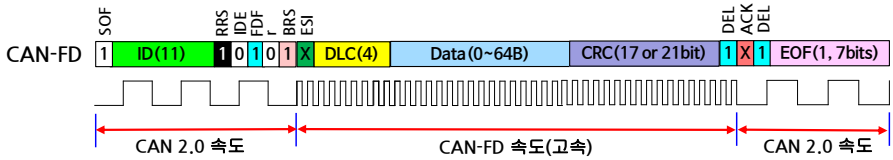
CAN-FD 특징과 동작 개념

CODEXPERT 임베디드 시스템 전문가그룹

❖ CAN-FD의 특징

- 속도 : CAN 2.0 ➔ 최대 500Kbps, CAN-FD ➔ 2/5/8 Mbps
- 데이터 : CAN 2.0 ➔ 0~8 바이트, CAN-FD ➔ 0~64 바이트
- CRC : CAN 2.0 ➔ 15 비트, CAN-FD ➔ 17 / 21 비트(신뢰성 향상)

❖ 가변 통신 속도 개념 ➔ 기존 CAN 2.0 버스에 호환성 유지 목적



❖ DLC 4bit로 데이터 바이트 수 결정

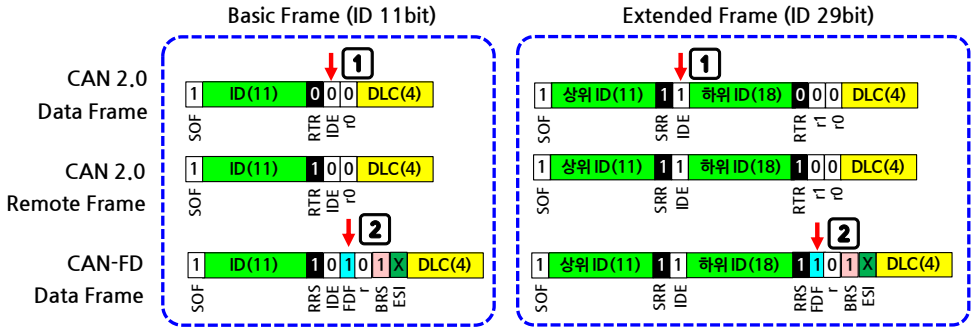
- CAN 2.0: DLC 값이 0 ~ 8이면 데이터 크기는 0 ~ 8B로 결정
- FD: 9 ➔ 12B, A ➔ 16B, B ➔ 20B , C ➔ 24B , D ➔ 32B , E ➔ 48B, F ➔ 64B

CAN 2.0과 CAN-FD의 Frame 구분

CODEXPERT 임베디드 시스템 전문가그룹

❖ Control 필드로 CAN-FD는 CAN 2.0 구분(단, FD는 Remote Frame 없음)

- 따라서 기존 CAN 2.0 Bus에 CAN-FD가 연결되어도 문제가 되지 않는다(호환성)



이름	길이(bits)	값	의미(Extended Data Frame)
Remote transmission request (RTR) (black) Remote Request Substitution (RRS) (black)	1 1	X 1	2.0: Data Frame(0), Remote Frame (1) FD: 항상 1로 고정
Identifier extension bit (IDE)	1	1	Base Frame (0), Extended Frame(1)
Flexible Data-rate Format (FDF) (cyan)	1	1	
Bit Rate Switch (BRS) (Pink)	1	1	이 비트 이후부터 고속 통신 속도로 변경됨
Error State Indicator (ESI) (green)	1	X	송신 노드 상태 : Active Error(0, 정상 포함), Passive Error(1)

CAN-XL 및 기타 연관 Protocols

CODEXPERT 임베디드 시스템 전문가그룹

❖ CAN-XL(eXtra Long): TCP/IP 데이터와 연동을 위한 구조

- 2018년 Bosche 특허, 2048바이트 페이로드 지원, 고속 데이터 전송 지원

❖ LIN(Local Interconnect Network)

- 자동차 제조사들이 컨소시엄을 구성하여 만든 통신 표준
- 주로 CAN의 하부 네트워크를 구성(몇 개의 지역 노드들을 하나로 묶는 개념)
  - 지역 노드들을 그룹화 용도 : 최대 20kbps로 저속이지만 높은 신뢰성 보장
  - Single Master - Multi Slave 구조로 구성 : 1개의 master는 최대 16개 slave와 연결 가능
  - 1가닥으로 통신을 수행하여 배선이 간단(신뢰성을 위하여 이중화 배선을 하기도 한다)

❖ 기타

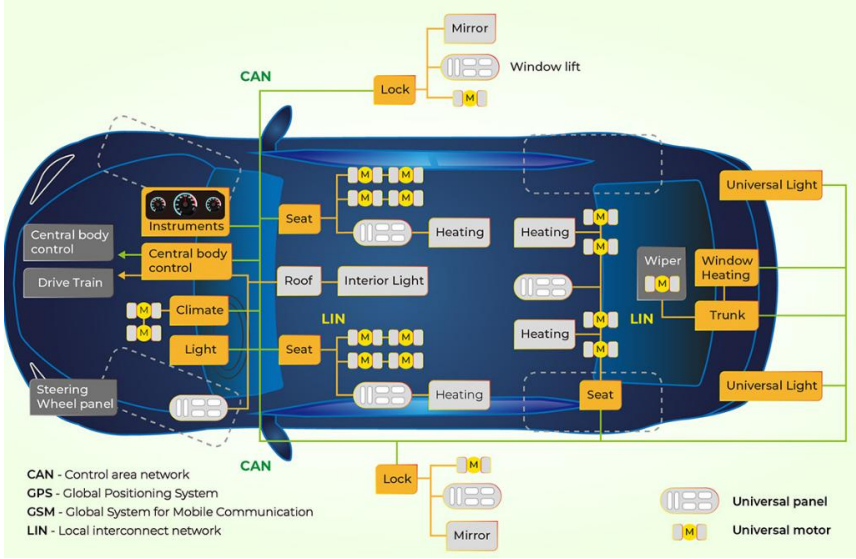
- CAN을 기본으로 많은 중간 계층의 통신 프로토콜들이 존재
- 용도에 따라(화물차, 특수 자동차 등) 변형된 많은 표준 통신 방식들이 존재
- CANopen, J1939(SAE), J2284(SAE), LeisureCAN, DeviceNet 등

 무단 복제, 전재, 발췌, 디지털화 금지

다양한 통신 표준의 사용

CODEXPERT 임베디드 시스템 전문가그룹

❖ 자동차 내부는 CAN뿐 아니라 다양한 통신 표준들이 혼합되어 사용된다



출처: https://www.winstar.com.tw/kr/can-bus-interface-communication

 무단 복제, 전재, 발췌, 디지털화 금지